



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Digital Assignment Lab - 1

Course Name :- Agile Development Process and Devops Lab

Course code :- ISWE406P

Name :- G SAI SANTHOSH

Reg-No:- 23MIS0227

Case Study 1: New Project Version Control Setup

Scenario: You are assigned to develop a Library Management System. Before coding, your team decides to enable version control.

Create a project folder and open it in VS Code

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

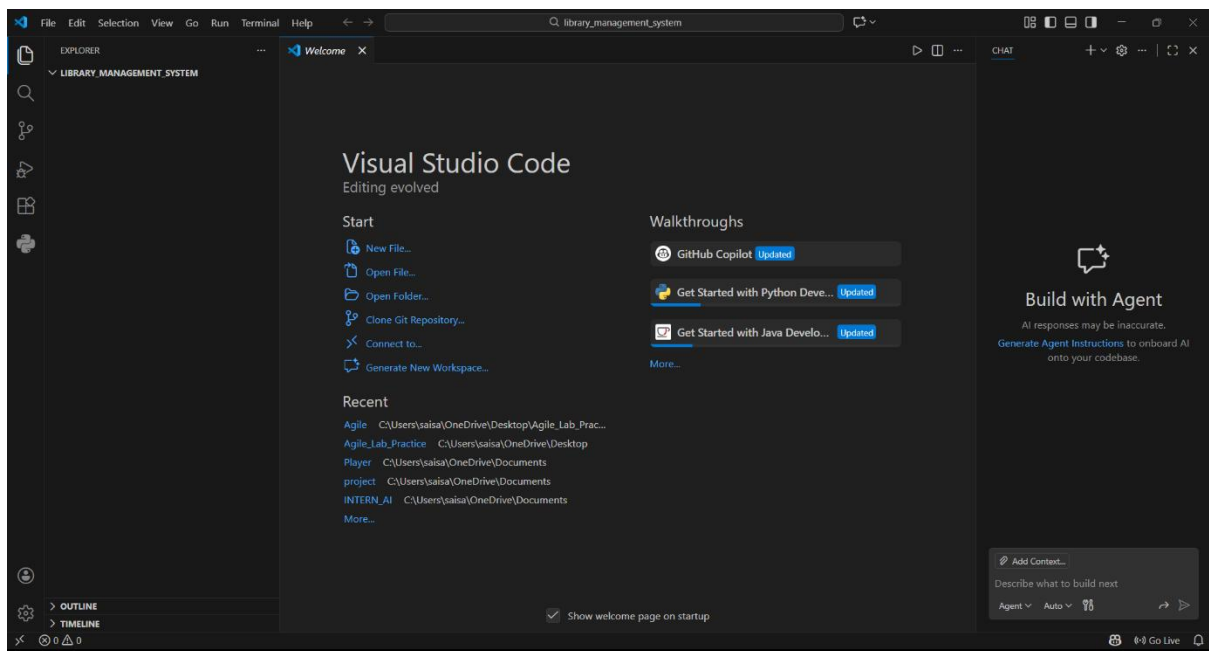
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\saisa\OneDrive\Desktop> mkdir library_management_system

Directory: C:\Users\saisa\OneDrive\Desktop

Mode                LastWriteTime         Length Name
----                -
d-----         08-01-2026         18:25         library_management_system

PS C:\Users\saisa\OneDrive\Desktop> |
```



Initialize a Git repository



The screenshot shows a terminal window with a dark background. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), and 'PORTS'. To the right of the tabs are icons for a terminal, a plus sign, a window, a trash can, and a menu. The terminal content shows two commands: the first is 'git init' which outputs 'Initialized empty Git repository in C:/Users/saia/OneDrive/Desktop/library_management_system/.git/', and the second is an empty prompt line.

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git init
Initialized empty Git repository in C:/Users/saia/OneDrive/Desktop/library_management_system/.git/
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Create a Python file and track it using Git

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> python -u "c:\Users\saisa\OneDrive\Desktop\library_management_system\main.py"
library management system
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    main.py

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git add main.py
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Commit the initial version

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git add main.py
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git commit -m "initial commit"
[master (root-commit) d95f886] initial commit
1 file changed, 1 insertion(+)
create mode 100644 main.py
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Case Study 2: Tracking Daily Code Changes

Scenario: You are working on a Student Registration Module. You make small updates daily and need to record them.

Modify an existing file

Check file differences

The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'LIBRARY_MANAGEMENT_SYSTEM' with a file 'main.py'. The main editor shows the content of 'main.py', which is a Python script for a student registration system. The script prompts for student ID, name, and department, then appends them to a list and prints the registered students. The Output pane at the bottom shows the command 'git diff' and its output, which displays the differences between the current version of 'main.py' and its previous commit. The diff shows that the file has been modified, with changes to the input prompts and the list of registered students. The status bar at the bottom indicates that the file is on the 'master' branch and has been modified.

```
main.py
1 students = []
2
3 sid = input("Enter Student ID: ")
4 name = input("Enter Name: ")
5 dept = input("Enter Department: ")
6
7 students.append([sid, name, dept])
8
9 print("Student Registered Successfully")
10 print("Registered Students:", students)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git diff
diff --git a/main.py b/main.py
index 6038cc8..17a871e 100644
--- a/main.py
+++ b/main.py
@@ -1,10 @@
-.print("library management system")
\ No newline at end of file
+students = []
+
+sid = input("Enter Student ID: ")
+name = input("Enter Name: ")
+dept = input("Enter Department: ")
+
+students.append([sid, name, dept])
+
+print("Student Registered Successfully")
+print("Registered Students:", students)
\ No newline at end of file
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Stage and commit changes

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   main.py

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git add .
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git commit -m "Modified main.py"
[master 0434c0c] Modified main.py
 1 file changed, 10 insertions(+), 1 deletion(-)
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

View commit history

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git log --oneline
0434c0c (HEAD -> master) Modified main.py
d95f886 initial commit
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

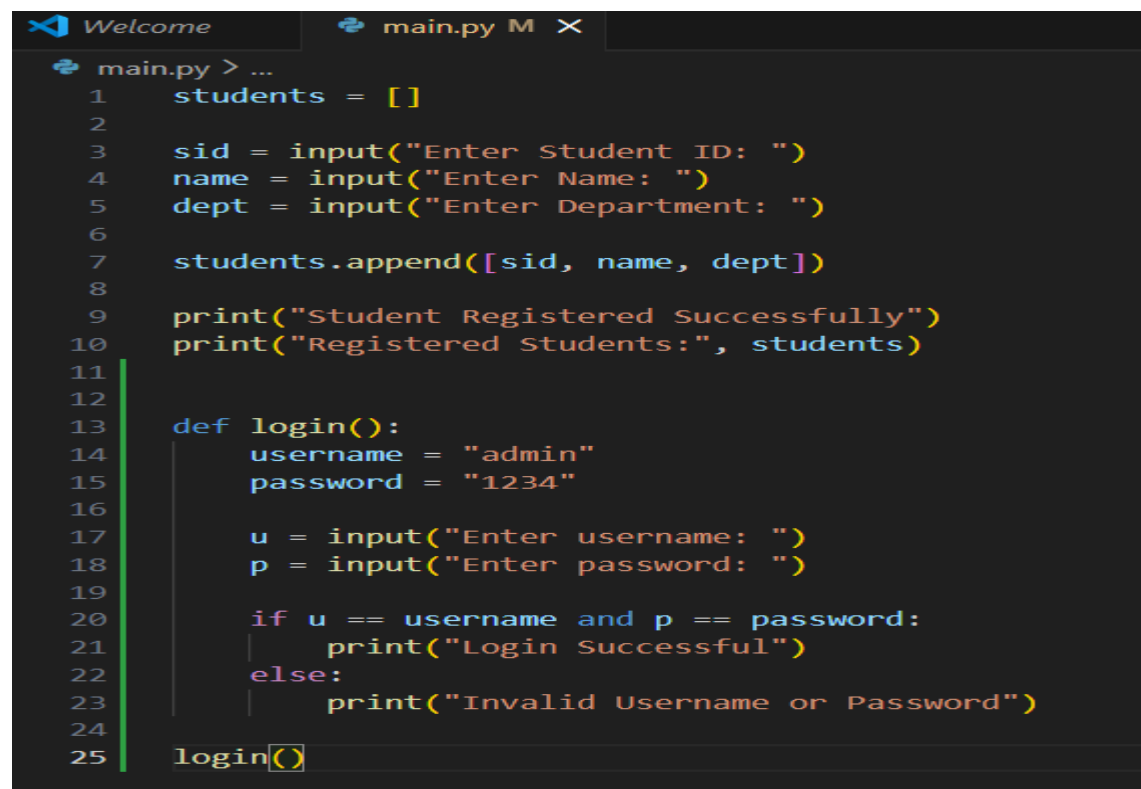
Case Study 3: Feature Development Using Branches

Scenario: A new Login feature must be developed without disturbing main code.

Create a new branch

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git branch login-feature
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git checkout login-feature
Switched to branch 'login-feature'
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Add login-related code



```
Welcome  main.py M X
main.py > ...
1  students = []
2
3  sid = input("Enter Student ID: ")
4  name = input("Enter Name: ")
5  dept = input("Enter Department: ")
6
7  students.append([sid, name, dept])
8
9  print("Student Registered Successfully")
10 print("Registered Students:", students)
11
12
13 def login():
14     username = "admin"
15     password = "1234"
16
17     u = input("Enter username: ")
18     p = input("Enter password: ")
19
20     if u == username and p == password:
21         print("Login Successful")
22     else:
23         print("Invalid Username or Password")
24
25 login()
```

Commit changes

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git add .
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git commit -m "Added login feature"
[login-feature c53b316] Added login feature
1 file changed, 16 insertions(+), 1 deletion(-)
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Switch between branches

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git switch master
Switched to branch 'master'
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Case Study 4: Merging Feature into Main Code

Scenario: The login feature is tested and approved.

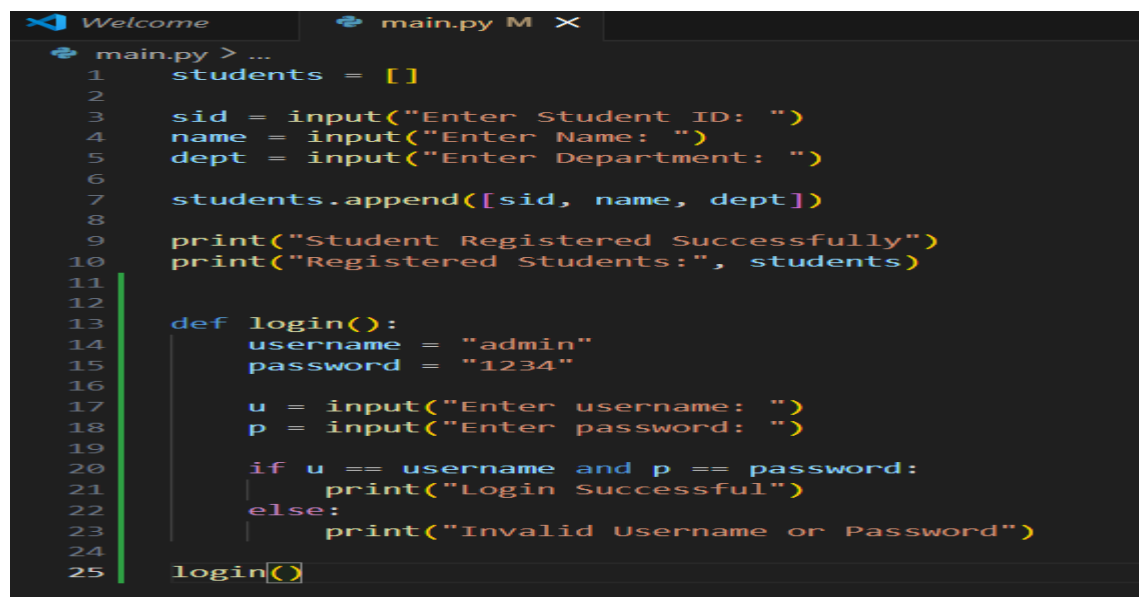
Switch to main branch

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git switch master
Switched to branch 'master'
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Merge feature branch

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git merge login-feature
Updating 0434c0c..c53b316
Fast-forward
 main.py | 17 ++++++++
1 file changed, 16 insertions(+), 1 deletion(-)
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Verify merged content

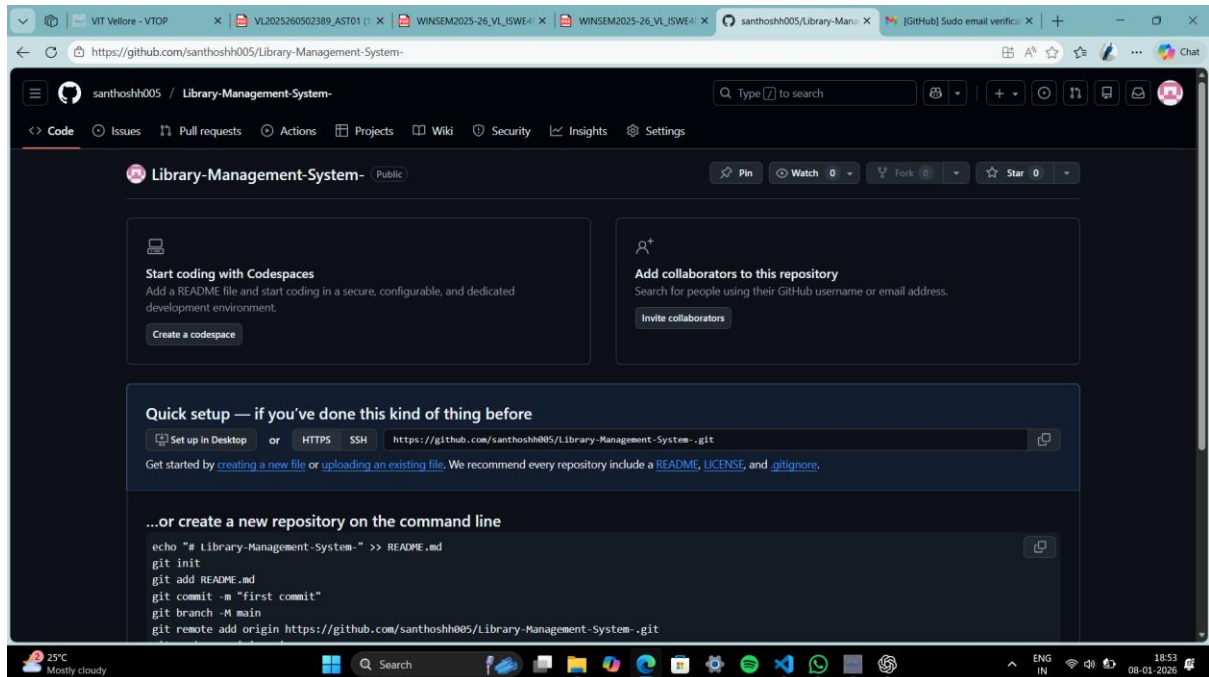


```
Welcome  main.py M X
main.py > ...
1  students = []
2
3  sid = input("Enter Student ID: ")
4  name = input("Enter Name: ")
5  dept = input("Enter Department: ")
6
7  students.append([sid, name, dept])
8
9  print("Student Registered Successfully")
10 print("Registered Students:", students)
11
12
13 def login():
14     username = "admin"
15     password = "1234"
16
17     u = input("Enter username: ")
18     p = input("Enter password: ")
19
20     if u == username and p == password:
21         print("Login Successful")
22     else:
23         print("Invalid Username or Password")
24
25 login()
```

Case Study 5: Working with Remote Repository

Scenario: The project must be shared using GitHub.

Create remote repository



Add remote origin

Push code to GitHub

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git remote add origin https://github.com/santhoshh005/Library-Management-System-.git
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git branch -M main
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 930 bytes | 310.00 KiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/santhoshh005/Library-Management-System-.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Case Study 6: Undoing Mistakes

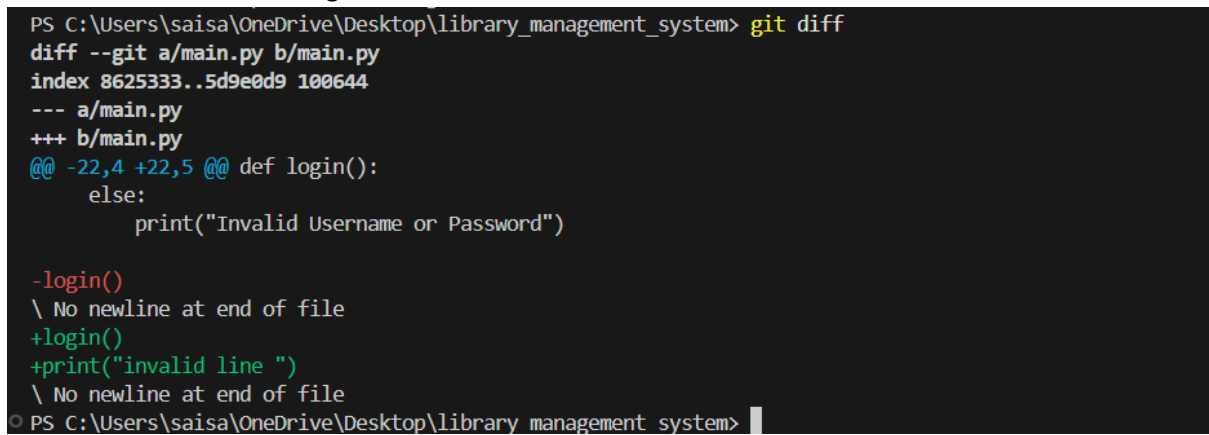
Scenario: A file is modified incorrectly.

View file changes



```
main.py > ...
13 def login():
16
17     u = input("Enter username: ")
18     p = input("Enter password: ")
19
20     if u == username and p == password:
21         print("Login Successful")
22     else:
23         print("Invalid Username or Password")
24
25     login()
26 | print("invalid line ")
```

Discard uncommitted changes



```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git diff
diff --git a/main.py b/main.py
index 8625333..5d9e0d9 100644
--- a/main.py
+++ b/main.py
@@ -22,4 +22,5 @@ def login():
     else:
         print("Invalid Username or Password")

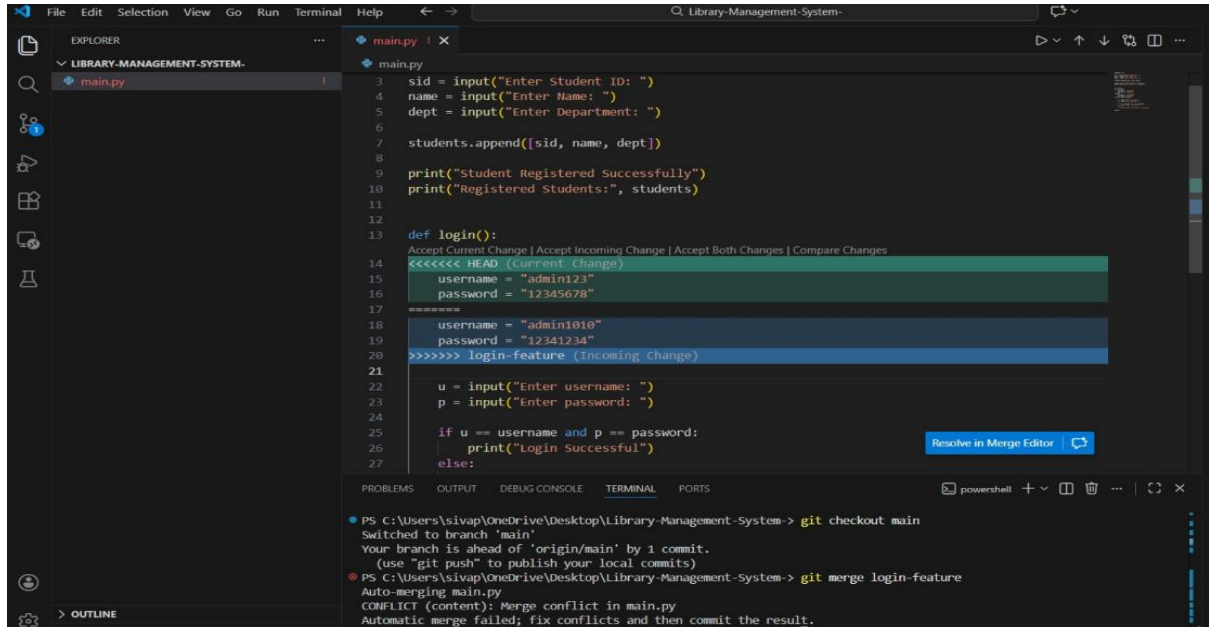
-login()
\ No newline at end of file
+login()
+print("invalid line ")
\ No newline at end of file
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```


Case Study 7: Handling Merge Conflicts

Scenario: Two developers edited the same file.

Merge branches

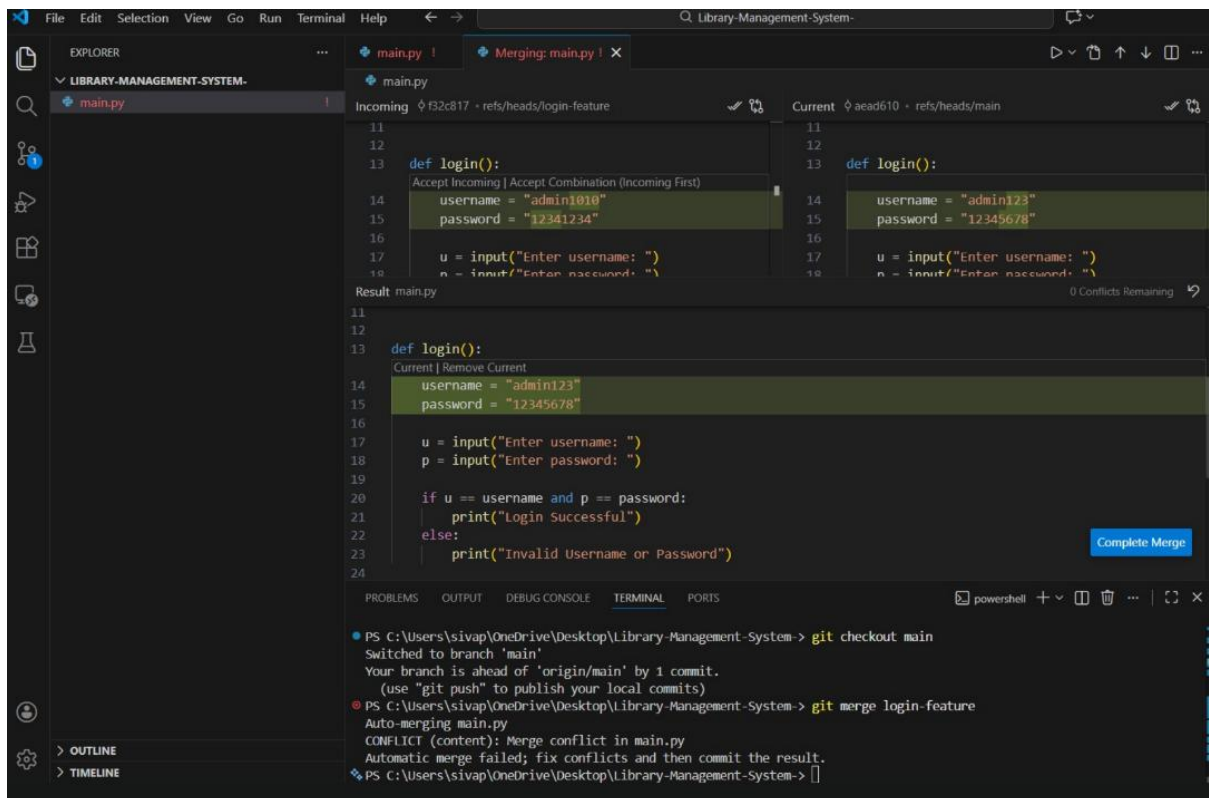
Resolve conflicts



```
3 sid = input("Enter Student ID: ")
4 name = input("Enter Name: ")
5 dept = input("Enter Department: ")
6
7 students.append([sid, name, dept])
8
9 print("Student Registered Successfully")
10 print("Registered Students:", students)
11
12
13 def login():
14     Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
15     <<<<<< HEAD (Current Change)
16     username = "admin123"
17     password = "12345678"
18     =====
19     username = "admin1010"
20     password = "12341234"
21     >>>>>> login-feature (Incoming Change)
22     u = input("Enter username: ")
23     p = input("Enter password: ")
24
25     if u == username and p == password:
26         print("Login Successful")
27     else:
```

PS C:\Users\sivap\OneDrive\Desktop\Library-Management-System> git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\sivap\OneDrive\Desktop\Library-Management-System> git merge login-feature
Auto-merging main.py
CONFLICT (content): Merge conflict in main.py
Automatic merge failed; fix conflicts and then commit the result.

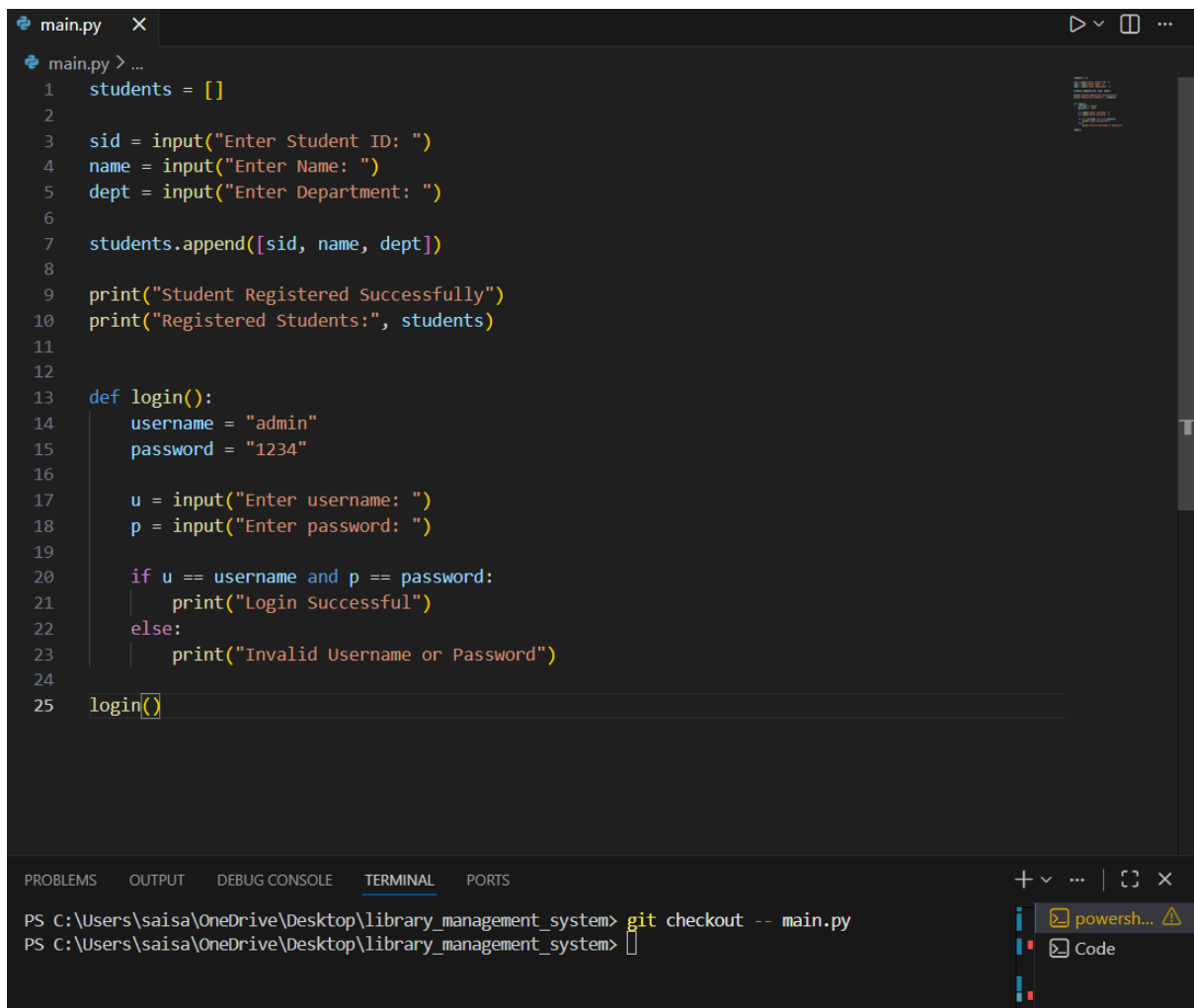
Commit resolved changes



```
11
12
13 def login():
14     Accept Incoming | Accept Combination (Incoming First)
15     username = "admin1010"
16     password = "12341234"
17
18     u = input("Enter username: ")
19     p = input("Enter password: ")
20
21     if u == username and p == password:
22         print("Login Successful")
23     else:
24         print("Invalid Username or Password")
25
```

Result main.py
Current | Remove Current
username = "admin123"
password = "12345678"
u = input("Enter username: ")
p = input("Enter password: ")
if u == username and p == password:
 print("Login Successful")
else:
 print("Invalid Username or Password")

PS C:\Users\sivap\OneDrive\Desktop\Library-Management-System> git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\sivap\OneDrive\Desktop\Library-Management-System> git merge login-feature
Auto-merging main.py
CONFLICT (content): Merge conflict in main.py
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\sivap\OneDrive\Desktop\Library-Management-System>



```
main.py X
main.py > ...
1  students = []
2
3  sid = input("Enter Student ID: ")
4  name = input("Enter Name: ")
5  dept = input("Enter Department: ")
6
7  students.append([sid, name, dept])
8
9  print("Student Registered Successfully")
10 print("Registered Students:", students)
11
12
13 def login():
14     username = "admin"
15     password = "1234"
16
17     u = input("Enter username: ")
18     p = input("Enter password: ")
19
20     if u == username and p == password:
21         print("Login Successful")
22     else:
23         print("Invalid Username or Password")
24
25 login()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

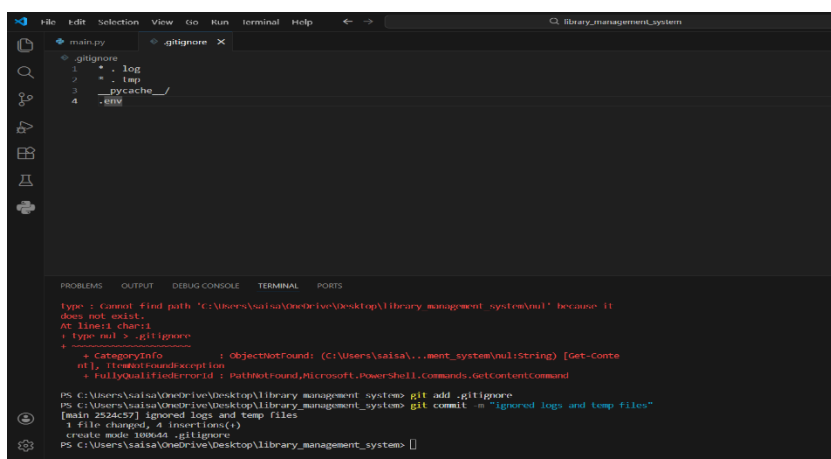
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git checkout -- main.py
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>

Case Study 8: Ignoring Unwanted Files

Scenario: Log and temporary files should not be tracked.

Create gitignore file

Add ignore patterns



```
main.py X .gitignore X
main.py
.gitignore
1  *.log
2  *.tmp
3  __pycache__
4  .DS_Store
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Type: Cannot find path 'C:\Users\saisa\OneDrive\Desktop\library_management_system\.gitignore' because it does not exist.
At line:1 char:1
+ type nul > .gitignore
+ ~~~~~
+ CategoryInfo : ObjectNotFound: (C:\Users\saisa\...ent_system\.gitignore:String) [Get-Content, HRESULT: 0x80070002] (PathNotFound, Microsoft.PowerShell.Commands.GetContentCommand)
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git add .gitignore
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git commit -m "ignored logs and temp files"
[main 2524c57] ignored logs and temp files
1 file changed, 4 insertions(+)
create mode 100644 .gitignore
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>

Case Study 9: Collaborating with Team

Scenario: A teammate pushed new changes.

Fetch latest changes

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git fetch origin
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 6 (delta 3), reused 5 (delta 2), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 523 bytes | 29.00 KiB/s, done.
From https://github.com/santhoshh005/Library-Management-System-
c53b316..aead610  main          -> origin/main
c53b316..b104e08  login-feature -> origin/login-feature
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Pull updates

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git pull origin main
From https://github.com/santhoshh005/Library-Management-System-
* branch          main          -> FETCH_HEAD
Already up to date.
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Resolve conflicts if any

Got No conflicts

Case Study 10: Final Project Release

Scenario: Project is ready for final submission.

Review commit history

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git log --oneline
d505f84 (HEAD -> main) ignored logs and temp files
2524c57 ignored logs and temp files
aead610 (origin/main, origin/HEAD) Modified credentials
2c3f811 (login-feature) modified login
c53b316 Added login feature
0434c0c Modified main.py
d95f886 initial commit
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

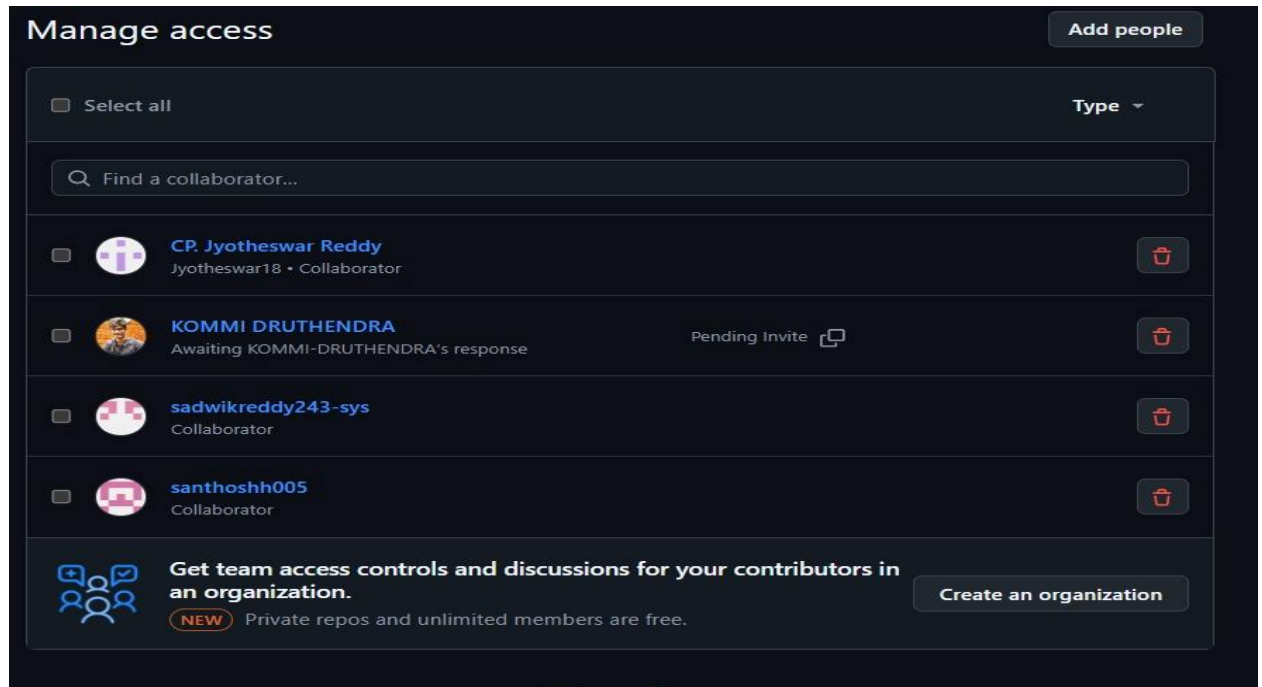
Push final code

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 579 bytes | 579.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/santhoshh005/Library-Management-System-.git
aead610..d505f84  main -> main
PS C:\Users\saisa\OneDrive\Desktop\library_management_system>
```

Prepare release version

```
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git tag v1.0
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> git push origin v1.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/santhoshh005/Library-Management-System-.git
 * [new tag]          v1.0 -> v1.0
PS C:\Users\saisa\OneDrive\Desktop\library_management_system> |
```

Class Assessment



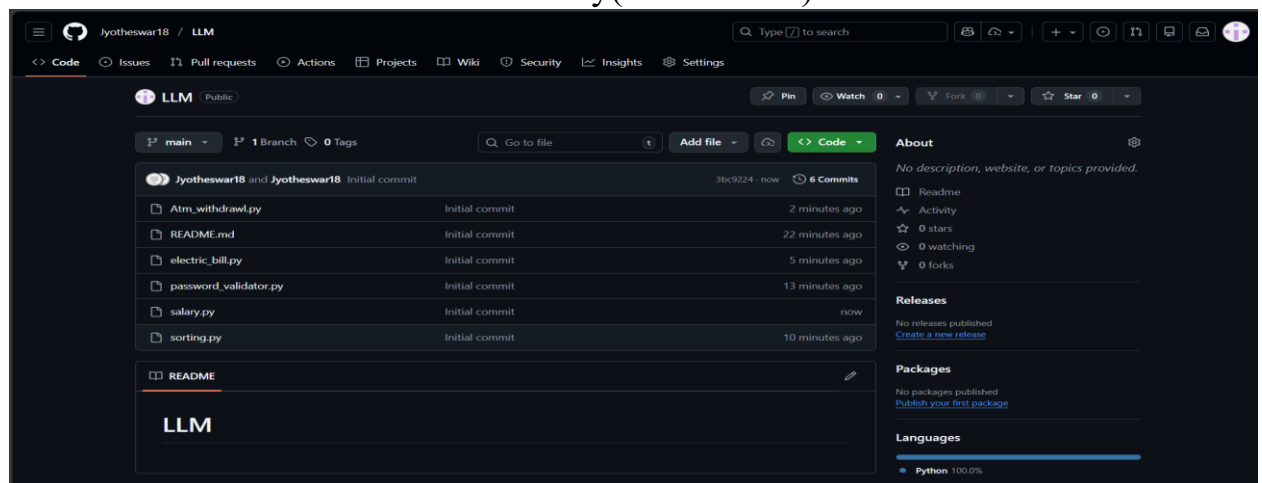
Team Members :-

Kommi Druthendra(23MIS0213)

G Sai Santhosh(23MIS0227)

CP Jyotheshwar Reddy(23MIS0097)

G Sadwik Reddy(23MIS0541)

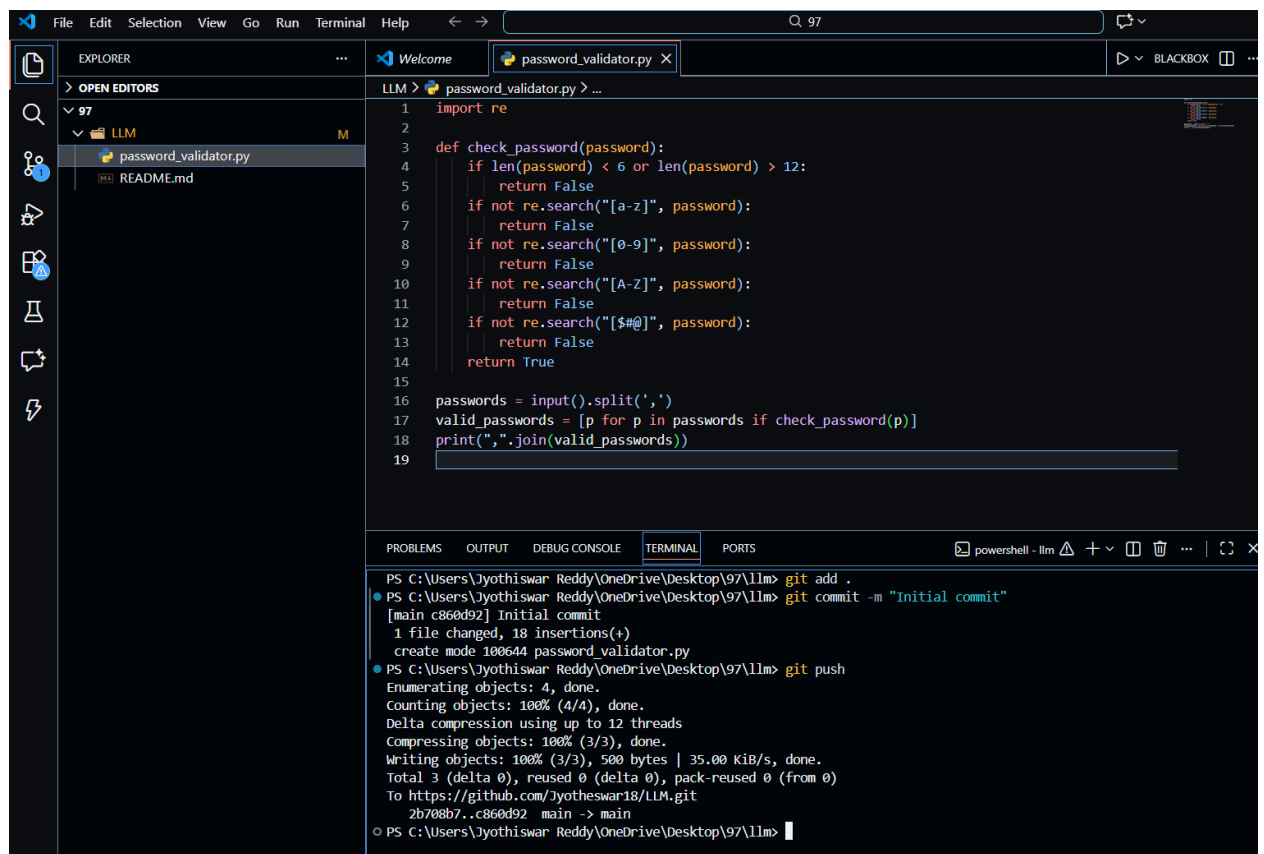


1) A website requires the users to input username and password to register. Write a program to check the validity of password input by users. Following are the criteria for checking the password:

1. At least 1 letter between [a-z]
 2. At least 1 number between [0-9]
 1. At least 1 letter between [A-Z]
 3. At least 1 character from [\$#@]
 4. Minimum length of transaction password: 6
 5. Maximum length of transaction password: 12
- Your program should accept a sequence of comma separated passwords and will check them according to the above criteria. Passwords that match the criteria are to be printed, each separated by a comma.

Example If the following passwords are given as input to the program: ABd1234@1,a F1#,2w3E*,2We3345
Then, the output of the program should be: ABd1234@1

CODE + OUTPUT :



```
File Edit Selection View Go Run Terminal Help
password_validator.py X
LLM > password_validator.py > ...
1 import re
2
3 def check_password(password):
4     if len(password) < 6 or len(password) > 12:
5         return False
6     if not re.search("[a-z]", password):
7         return False
8     if not re.search("[0-9]", password):
9         return False
10    if not re.search("[A-Z]", password):
11        return False
12    if not re.search("[$#@]", password):
13        return False
14    return True
15
16 passwords = input().split(',')
17 valid_passwords = [p for p in passwords if check_password(p)]
18 print(",".join(valid_passwords))
19

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git add .
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git commit -m "Initial commit"
[main c860d92] Initial commit
1 file changed, 18 insertions(+)
create mode 100644 password_validator.py
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 500 bytes | 35.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Jyothiswar18/LLM.git
2b708b7..c860d92 main -> main
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm>
```

2) You are required to write a program to sort the (name, age, height) tuples by ascending order where name is string, age and height are numbers. The tuples are input by console. The sort criteria is:

1: Sort based on name;

2: Then sort based on age;

3: Then sort by score. The priority is that name > age > score. If the following tuples are given as input to the program:

Tom,19,80 John,20,90 Jony,17,91 Jony,17,93 Json,21,85

Then, the output of the program should be: [('John', '20', '90'), ('Jony', '17', '91'), ('Jony', '17', '93'), ('Json', '21', '85'), ('Tom', '19', '80')]

CODE + OUTPUT :

The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a folder named 'LLM' containing 'password_validator.py', 'README.md', and 'sorting.py'. The code editor shows the 'sorting.py' file with the following code:

```
1 items = []
2 while True:
3     try:
4         line = input()
5         if not line:
6             break
7         items.append(tuple(line.split(",")))
8     except:
9         break
10
11 items.sort(key=lambda x: (x[0], int(x[1]), int(x[2])))
12 print(items)
13
```

The terminal at the bottom shows the execution of the script using PowerShell. The output is:

```
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git add .
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git commit -m "Initial commit"
[main 6c4c72e] Initial commit
1 file changed, 12 insertions(+)
create mode 100644 sorting.py
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 488 bytes | 162.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Jyothiswar18/LLM.git
c860d92..6c4c72e main -> main
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm>
```

3) An electricity board calculates charges based on usage. Write a program that: • Takes number of units consumed •

Calculates bill:

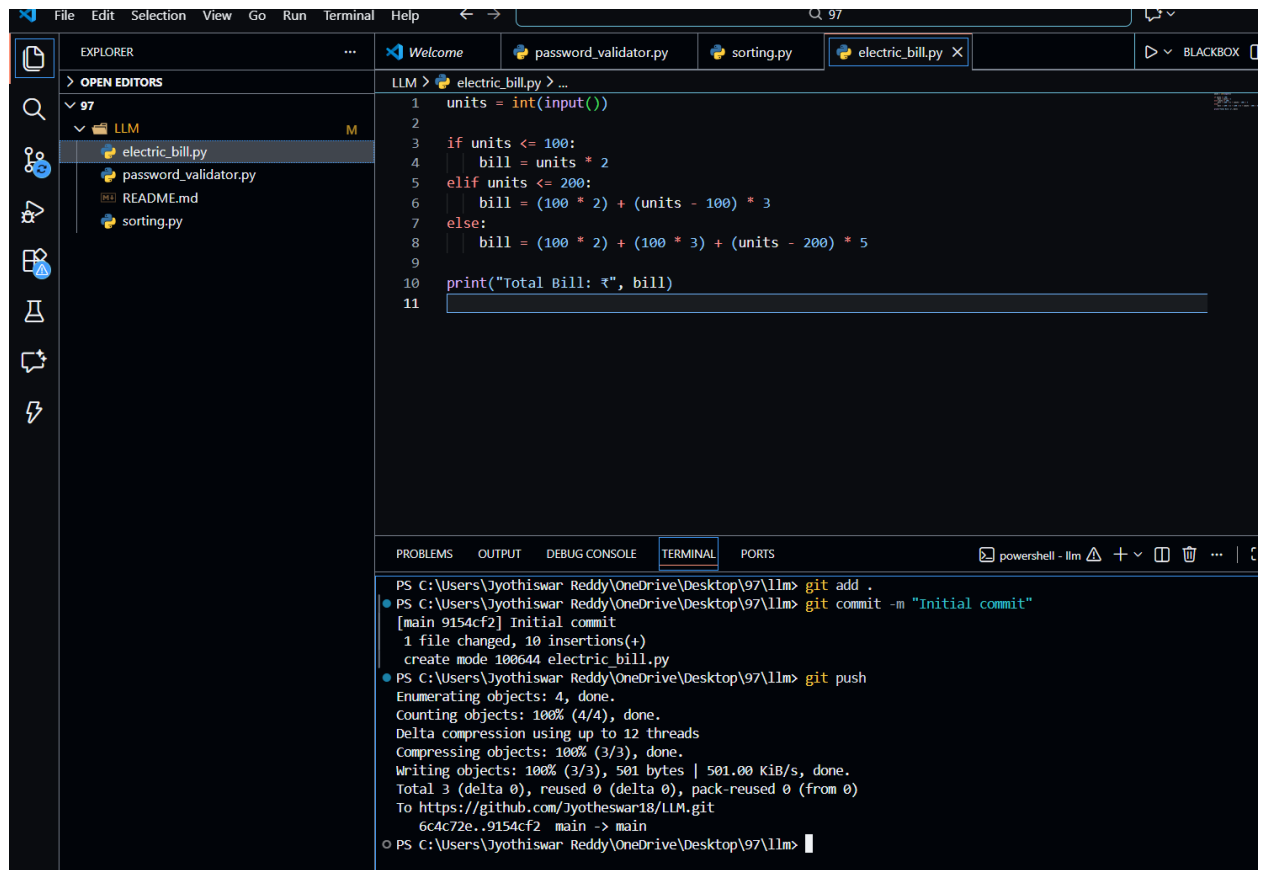
First 100 units → ₹2/unit

Next 100 units → ₹3/unit

Remaining → ₹5/unit

Displays total bill

CODE + OUTPUT :



The screenshot shows a Visual Studio Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a folder named 'LLM' containing 'electric_bill.py', 'password_validator.py', 'README.md', and 'sorting.py'. The editor window shows the code for 'electric_bill.py'.

```
1 units = int(input())
2
3 if units <= 100:
4     bill = units * 2
5 elif units <= 200:
6     bill = (100 * 2) + (units - 100) * 3
7 else:
8     bill = (100 * 2) + (100 * 3) + (units - 200) * 5
9
10 print("Total Bill: ₹", bill)
11
```

The terminal shows the output of the program:

```
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git add .
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git commit -m "Initial commit"
[main 9154cf2] Initial commit
1 file changed, 10 insertions(+)
create mode 100644 electric_bill.py
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 501 bytes | 501.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Jyothiswar18/LLM.git
6c4c72e..9154cf2 main -> main
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm>
```

4) An ATM machine allows withdrawals only if sufficient balance is available.

Write a program that: Stores a user's balance

- Takes withdrawal amount as input
- Checks: Amount must be a multiple of 100 o Balance must be sufficient
- Deducts amount and displays updated balance

CODE + OUTPUT :

```
File Edit Selection View Go Run Terminal Help
Atm_withdrawl.py x BLACKBOX

EXPLORED
OPEN EDITORS
97
LLM
Atm_withdrawl.py
electric_bill.py
password_validator.py
README.md
sorting.py

LLM > Atm_withdrawl.py > ...
1 balance = int(input())
2 withdraw = int(input())
3
4 if withdraw % 100 != 0:
5     print("Amount must be multiple of 100")
6 elif withdraw > balance:
7     print("Insufficient balance")
8 else:
9     balance -= withdraw
10    print("Updated Balance:", balance)
11

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell - llm + - - - |

PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git add .
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git commit -m "Initial commit"
[main 89aa328] Initial commit
1
cr Focus folder in explorer (ctrl + click) py
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 439 bytes | 439.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Jyothiswar18/LLM.git
9154cf2..89aa328 main -> main
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm>
```


5) A company calculates monthly salary based on basic pay. Write a program that:

- Accepts basic salary
- Calculates: HRA = 20% of basic
DA = 10% of basic
Tax = 5% of total salary
- Displays net salary

CODE + OUTPUT :

The screenshot shows a Visual Studio Code editor window with the following components:

- EXPLORER:** Displays a folder named 'LLM' containing files: 'Atm_withdrawl.py', 'electric_bill.py', 'password_validator.py', 'README.md', 'salary.py', and 'sorting.py'.
- EDITOR:** The active file is 'salary.py', containing the following Python code:

```
1 basic = float(input())
2 hra = 0.2 * basic
3 da = 0.1 * basic
4 total = basic + hra + da
5 tax = 0.05 * total
6 net_salary = total - tax
7 print("Net Salary:", net_salary)
8
```
- TERMINAL:** Shows the execution of the program and subsequent git commands:

```
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git add .
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git commit -m "Initial commit"
[main 3bc9224] Initial commit
1 file changed, 7 insertions(+)
create mode 100644 salary.py
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm> git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 399 bytes | 399.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Jyothiswar18/LLM.git
89aa328..3bc9224 main -> main
PS C:\Users\Jyothiswar Reddy\OneDrive\Desktop\97\llm>
```